

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д. О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 3

ОРГАНИЗАЦИЯ ПОЛУЧЕНИЯ И ВЫВОДА ДАННЫХ С УДАЛЕННОГО
ОБОРУДОВАНИЯ

по курсу:

ИНТЕРНЕТ ВЕЩЕЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 4326

1 Цель работы

Цель работы: приобретение навыков организации получения данных с удаленного оборудования.

2 Задание

Реализовать эмуляторы вещей для всех классов, созданных в предыдущей лабораторной. Разработать один или несколько интерфейсов, в которые будут передаваться данные от эмуляторов. В рамках данной лабораторной работы необходимо передать в интерфейс только данные для мониторинга. Передача команд, которые осуществляют управление вещью, будет рассмотрена в следующей лабораторной. Для каждой вещи необходимо реализовать собственный аякс-запрос и собственную функцию подключения.

Вариант задания: «Умная теплица».

3 Ход выполнения

Для каждого класса устройства были реализованы методы эмуляции состояния и метод подключения, возвращающий данные для мониторинга. В составе модели были использованы датчики, исполнительные устройства, главный контроллер и модуль базы данных.

Код классов и методов эмуляции изображен на рисунках 1–5.

```
7 class Device(ABC):
8 >     def __init__(self, device_id: int, name: str, status: bool = False):...
12
13 >     def turn_on(self):...
16
17 >     def turn_off(self):...
20
21 >     def get_status(self) → bool:...
26
27     @abstractmethod
28     def connect(self) → Dict[str, Union[int, float, str, bool]]:
29         """Возвращает данные для мониторинга."""
30
```

Рисунок 1 — Абстрактный метод connect

```

32 class Sensor(Device):
33 >     def __init__(self, device_id: int, name: str, sensor_type: str, unit: str):...
38
39 >     def measure(self):...
44
45 >     def calibrate(self, value: float):...
50
51     def emulate(self):
52         ranges = {
53             "temperature": (18.0, 30.0),
54             "humidity": (40.0, 90.0),
55             "soil_moisture": (25.0, 80.0),
56         }
57         low, high = ranges.get(self.type, (0.0, 100.0))
58         self.value = round(random.uniform(low, high), 1)
59
60     def connect(self) → Dict[str, Union[int, float, str, bool]]:
61         self.emulate()
62         print(
63             f"[LOG] Подключение к датчику {self.name} выполнено, новое значение: {self.value} {self.unit}"
64         )
65         return {
66             "id": self.id,
67             "name": self.name,
68             "type": self.type,
69             "unit": self.unit,
70             "value": self.value,
71             "status": self.status,
72         }
73

```

Рисунок 2 — Реализованные методы emulate и connect класса Sensor

```

75 class Actuator(Device):
76 >     def __init__(...
78 >     ):...
82
83 >     def apply_action(self, action: str):...
87
88 >     def set_power(self, level: float):...
93
94     def emulate(self):
95         self.power = round(random.uniform(20.0, 250.0), 1)
96         self.status = random.choice([True, False])
97
98     def connect(self) → Dict[str, Union[int, float, str, bool]]:
99         self.emulate()
100         print(
101             f"[LOG] Подключение к исполнительному устройству {self.name} выполнено, мощность: {self.power} Вт, статус: {self.status}"
102         )
103         return {
104             "id": self.id,
105             "name": self.name,
106             "type": self.type,
107             "power": self.power,
108             "status": self.status,
109         }

```

Рисунок 3 — Реализованные методы emulate и connect класса Actuator

```

112 class MainControlUnit:
113 >     def __init__(self):...
117
118 >     def process_data(self, sensor_data: Dict[int, float]):...
120
121 >     def send_command(self, actuator: Actuator, command: str):...
126
127 >     def check_alerts(self):...
133
134     def emulate(self):
135         self.mode = random.choice(["auto", "manual"])
136         if random.random() > 0.6:
137             self.alerts = [
138                 random.choice(["Перегрев", "Низкая влажность", "Ошибка насоса"])
139             ]
140         else:
141             self.alerts = []
142
143     def connect(self) → Dict[str, Union[str, int]]:
144         self.emulate()
145         print(
146             f"[LOG] Подключение к контроллеру выполнено, режим: {self.mode}, количество тревог: {len(self.alerts)}"
147         )
148         return {
149             "mode": self.mode,
150             "alerts_count": len(self.alerts),
151         }

```

Рисунок 4 — Реализованные методы emulate и connect класса MainControlUnit

```

154 class Database:
155 >     def __init__(self, connection_string: str, storage_path: str):...
158
159 >     def save_reading(self, sensor_id: int, value: float, timestamp: datetime):...
163
164 >     def save_event(self, device_id: int, action: str, timestamp: datetime):...
168
169 >     def get_history(...
171 >     ):...
177
178     def emulate(self) → Dict[str, Union[str, int]]:
179         return {
180             "connection": "ok",
181             "storage_path": self.storage_path,
182             "records_today": random.randint(30, 300),
183         }
184
185     def connect(self) → Dict[str, Union[str, int]]:
186         data = self.emulate()
187         print(
188             f"[LOG] Подключение к БД выполнено, записей за сегодня: {data['records_today']}, путь: {data['storage_path']}"
189         )
190         return data
191

```

Рисунок 5 — Реализованные методы emulate и connect класса Database

Далее была доработана серверная часть приложения на Flask. Было реализовано отображение основной страницы интерфейса через шаблон, а также созданы отдельные маршруты подключения для каждого типа устройства. Для каждого маршрута был обеспечен возврат данных в формате JSON, что позволило организовать корректный обмен между сервером и клиентской частью.

Код серверных маршрутов и инициализации объектов изображен на рисунках 6–7.

```
9  temperature_sensor = Sensor(1, "Датчик температуры", "temperature", "°C")
10 humidity_sensor = Sensor(2, "Датчик влажности воздуха", "humidity", "%")
11 soil_sensor = Sensor(3, "Датчик влажности почвы", "soil_moisture", "%")
12 fan = Actuator(101, "Вентилятор", "fan", 120.0)
13 pump = Actuator(102, "Насос полива", "pump", 80.0)
14 controller = MainControlUnit()
15 database = Database("sqlite:///greenhouse.db", "./data")
16
17 temperature_sensor.turn_on()
18 humidity_sensor.turn_on()
19 soil_sensor.turn_on()
20 fan.turn_on()
21 pump.turn_on()
22
```

Рисунок 6 — Инициализация объектов системы

```
24 @app.route("/")
25 def index():
26     return render_template("dashboard.html")
27
28
29 @app.route("/connect/temperature")
30 def connect_temperature():
31     return jsonify(temperature_sensor.connect())
32
33
34 @app.route("/connect/humidity")
35 def connect_humidity():
36     return jsonify(humidity_sensor.connect())
37
38
39 @app.route("/connect/soil")
40 def connect_soil():
41     return jsonify(soil_sensor.connect())
42
43
44 @app.route("/connect/fan")
45 def connect_fan():
46     return jsonify(fan.connect())
47
48
49 @app.route("/connect/pump")
50 def connect_pump():
51     return jsonify(pump.connect())
52
53
54 @app.route("/connect/controller")
55 def connect_controller():
56     return jsonify(controller.connect())
57
58
59 @app.route("/connect/database")
60 def connect_database():
61     return jsonify(database.connect())
```

Рисунок 7 — Определение серверных маршрутов

После этого была сформирована структура клиентской части с разделением на шаблоны, скрипты и стили. В HTML-шаблоне были добавлены поля мониторинга для всех эмулируемых сущностей «умной теплицы». Для удобства визуального контроля данные были сгруппированы по карточкам.

Код HTML-шаблона интерфейса изображен на рисунке 8

```
1 <!DOCTYPE html>
2 <html lang="ru">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>Умная теплица</title>
8   <link rel="stylesheet" href="{{ url_for('static', filename='css/main.css') }}">
9   <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
10  <script src="{{ url_for('static', filename='js/index.js') }}" defer></script>
11 </head>
12
13 <body>
14   <main class="container">
15     <section class="card main">
16       <h2>Агроном (оператор)</h2>
17       <div class="grid">
18         <article class="card">
19           <h2>Датчик температуры</h2>
20           <input id="temperature_value" type="text" readonly>
21         </article>
22         <article class="card">
23           <h2>Датчик влажности воздуха</h2>
24           <input id="humidity_value" type="text" readonly>
25         </article>
26         <article class="card">
27           <h2>Датчик влажности почвы</h2>
28           <input id="soil_value" type="text" readonly>
29         </article>
30         <article class="card">
31           <h2>Вентилятор</h2>
32           <input id="fan_power" type="text" readonly>
33         </article>
34         <article class="card">
35           <h2>Насос</h2>
36           <input id="pump_power" type="text" readonly>
37         </article>
38         <article class="card">
39           <h2>Контроллер</h2>
40           <input id="controller_mode" type="text" readonly>
41           <input id="controller_alerts" type="text" readonly>
42         </article>
43         <article class="card">
44           <h2>База данных</h2>
45           <input id="db_connection" type="text" readonly>
46           <input id="db_records" type="text" readonly>
47         </article>
48       </div>
49     </section>
50   </main>
51 </body>
52
53 </html>
```

Рисунок 8 — HTML-код интерфейса

На стороне JavaScript были реализованы отдельные AJAX-запросы для каждой вещи, что соответствует требованиям задания. Было выполнено

периодическое обращение к серверу с интервалом 1 секунда, что обеспечило регулярное обновление данных мониторинга в полях интерфейса. Подписи к показаниям были приведены к единому русскоязычному виду.

Пример кода клиентских AJAX-функций изображен на рисунке 9. На рисунке 10 приведена основная периодическая функция обновления интерфейса.

```
1  function getTemperatureData() {  
2      $.ajax({  
3          type: "GET",  
4          url: "/connect/temperature",  
5          dataType: "json",  
6          contentType: "application/json",  
7          data: {},  
8          success: function (response) {  
9              document.getElementById("temperature_value").value = response.value + " " + response.unit;  
10         }  
11     });  
12 }  
13
```

Рисунок 9 — Пример AJAX-запроса данных температурного датчика

```
94  function updateDashboard() {  
95      getTemperatureData();  
96      getHumidityData();  
97      getSoilData();  
98      getFanData();  
99      getPumpData();  
100     getControllerData();  
101     getDatabaseData();  
102 }  
103  
104 setInterval(updateDashboard, 1000);  
105 updateDashboard();  
106
```

Рисунок 10 — Периодическая функция обновления интерфейса

Далее было выполнено оформление интерфейса с использованием CSS, а также структурирование стилевых параметров через переменные (дизайн-токены) для базовых визуальных характеристик. Это позволило повысить читаемость и поддерживаемость оформления.

Фрагмент кода таблицы стилей изображен на рисунке 11.

```

1  :root {
2    --font-family-base: Arial, sans-serif;
3
4    --color-bg: #f4f8f5;
5    --color-text: #182d4a;
6    --color-muted: #4f6155;
7    --color-surface: #ffffff;
8    --color-surface-alt: #e3ecf7;
9    --color-border: #b2c6e2;
10   --color-accent: #182d4a;
11
12   --radius-sm: 8px;
13   --radius-md: 24px;
14
15   --shadow-card: 0 2px 6px rgba(31, 43, 34, 0.08);
16 }
17
18 body {
19   margin: 0;
20   font-family: var(--font-family-base);
21   background: var(--color-bg);
22   color: var(--color-text);
23 }
24
25 .container {
26   max-width: 600px;
27   margin-inline: auto;
28   padding: 24px;
29 }
30
31 .grid {
32   display: grid;
33   grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
34   gap: 16px;
35   margin-top: 18px;
36 }

```

Рисунок 11 — Фрагмент кода CSS-стилей интерфейса

Реализованный пользовательский интерфейс приведен на рисунке 12.

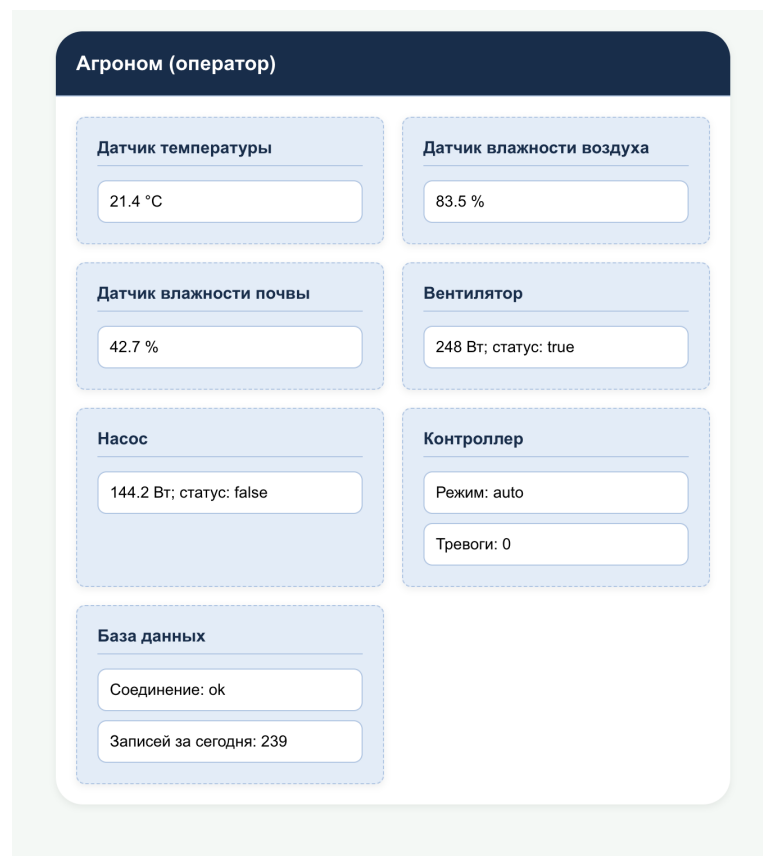


Рисунок 12 — Реализованный интерфейс в браузере

4 Выводы

В результате выполнения лабораторной работы была достигнута поставленная цель по освоению принципов организации получения и вывода данных с удаленного оборудования. Была реализована архитектура мониторинга для системы умной теплицы, в которой обмен между серверной и клиентской частями осуществляется через GET-запросы и JSON-ответы. Для всех ключевых сущностей предметной области были созданы эмуляторы, обеспечивающие формирование актуальных параметров состояния, что позволило смоделировать работу реальной IoT-системы без подключения физического оборудования.

В ходе работы была подтверждена корректность взаимодействия между компонентами: серверные маршруты стабильно возвращают данные, клиентская часть регулярно запрашивает их и отображает в интерфейсе в режиме периодического обновления. Разделение проекта на серверную логику, шаблоны, скрипты и стили обеспечило понятную структуру и удобство сопровождения. Полученный результат демонстрирует практическую готовность системы к дальнейшему развитию.

ПРИЛОЖЕНИЕ

Исходный код

Исходный код приложения также доступен по ссылке:

<https://github.com/grigoriytomczuk/smart-greenhouse>.

Листинг 1 — app.py

```
import logging

from flask import Flask, jsonify, render_template

from devices import Actuator, Database, MainControlUnit, Sensor

app = Flask(__name__)

temperature_sensor = Sensor(1, "Датчик температуры", "temperature", "°C")
humidity_sensor = Sensor(2, "Датчик влажности воздуха", "humidity", "%")
soil_sensor = Sensor(3, "Датчик влажности почвы", "soil_moisture", "%")
fan = Actuator(101, "Вентилятор", "fan", 120.0)
pump = Actuator(102, "Насос полива", "pump", 80.0)
controller = MainControlUnit()
database = Database("sqlite:///greenhouse.db", "./data")

temperature_sensor.turn_on()
humidity_sensor.turn_on()
soil_sensor.turn_on()
fan.turn_on()
pump.turn_on()

@app.route("/")
def index():
    return render_template("dashboard.html")

@app.route("/connect/temperature")
def connect_temperature():
    return jsonify(temperature_sensor.connect())

@app.route("/connect/humidity")
def connect_humidity():
    return jsonify(humidity_sensor.connect())

@app.route("/connect/soil")
def connect_soil():
    return jsonify(soil_sensor.connect())

@app.route("/connect/fan")
def connect_fan():
    return jsonify(fan.connect())

@app.route("/connect/pump")
def connect_pump():
    return jsonify(pump.connect())
```

```

@app.route("/connect/controller")
def connect_controller():
    return jsonify(controller.connect())

@app.route("/connect/database")
def connect_database():
    return jsonify(database.connect())

if __name__ == "__main__":
    logging.getLogger("werkzeug").setLevel(logging.ERROR)
    app.run(debug=True)

```

Листинг 2 — devices.py

```

import random
from abc import ABC, abstractmethod
from datetime import datetime
from typing import Dict, List, Optional, Union

class Device(ABC):
    def __init__(self, device_id: int, name: str, status: bool = False):
        self.id = device_id
        self.name = name
        self.status = status

    def turn_on(self):
        self.status = True
        print(f"[LOG] {self.__class__.__name__} {self.name} (ID: {self.id}) включён")

    def turn_off(self):
        self.status = False
        print(f"[LOG] {self.__class__.__name__} {self.name} (ID: {self.id}) выключен")

    def get_status(self) -> bool:
        print(
            f"[LOG] Запрос статуса {self.__class__.__name__} {self.name} (ID: {self.id}) -> {self.status}"
        )
        return self.status

    @abstractmethod
    def connect(self) -> Dict[str, Union[int, float, str, bool]]:
        """Возвращает данные для мониторинга."""

class Sensor(Device):
    def __init__(self, device_id: int, name: str, sensor_type: str, unit: str):
        super().__init__(device_id, name)
        self.type = sensor_type
        self.unit = unit
        self.value: float = 0.0

    def measure(self):
        self.value = 22.5
        print(
            f"[LOG] Датчик {self.name} (ID: {self.id}) измерил: {self.value}"

```

```

{self.unit}"
    )

    def calibrate(self, value: float):
        self.value = value
        print(
            f"[LOG] Датчик {self.name} (ID: {self.id}) откалиброван на значение
{value} {self.unit}"
        )

    def emulate(self):
        ranges = {
            "temperature": (18.0, 30.0),
            "humidity": (40.0, 90.0),
            "soil_moisture": (25.0, 80.0),
        }
        low, high = ranges.get(self.type, (0.0, 100.0))
        self.value = round(random.uniform(low, high), 1)

    def connect(self) -> Dict[str, Union[int, float, str, bool]]:
        self.emulate()
        print(
            f"[LOG] Подключение к датчику {self.name} выполнено, новое значение:
{self.value} {self.unit}"
        )
        return {
            "id": self.id,
            "name": self.name,
            "type": self.type,
            "unit": self.unit,
            "value": self.value,
            "status": self.status,
        }

class Actuator(Device):
    def __init__(
        self, device_id: int, name: str, actuator_type: str, power: float = 0.0
    ):
        super().__init__(device_id, name)
        self.type = actuator_type
        self.power = power

    def apply_action(self, action: str):
        print(
            f"[LOG] Исполнительное устройство {self.name} (ID: {self.id}) выполняет
действие: {action}"
        )

    def set_power(self, level: float):
        self.power = level
        print(
            f"[LOG] Мощность устройства {self.name} (ID: {self.id}) установлена на
{level} Вт"
        )

    def emulate(self):
        self.power = round(random.uniform(20.0, 250.0), 1)
        self.status = random.choice([True, False])

    def connect(self) -> Dict[str, Union[int, float, str, bool]]:
        self.emulate()

```

```

        print(
            f"[LOG] Подключение к исполнительному устройству {self.name} выполнено,
            мощность: {self.power} Вт, статус: {self.status}"
        )
        return {
            "id": self.id,
            "name": self.name,
            "type": self.type,
            "power": self.power,
            "status": self.status,
        }

class MainControlUnit:
    def __init__(self):
        self.mode = "auto"
        self.schedule: Dict[str, str] = {}
        self.alerts: List[str] = []

    def process_data(self, sensor_data: Dict[int, float]):
        print(f"[LOG] Контроллер обрабатывает данные с датчиков: {sensor_data}")

    def send_command(self, actuator: Actuator, command: str):
        print(
            f"[LOG] Контроллер отправляет команду '{command}' устройству
            {actuator.name} (ID: {actuator.id})"
        )
        actuator.apply_action(command)

    def check_alerts(self):
        print("[LOG] Контроллер проверяет наличие тревог...")
        if self.alerts:
            print(f"[LOG] Активные тревоги: {self.alerts}")
        else:
            print("[LOG] Тревог нет")

    def emulate(self):
        self.mode = random.choice(["auto", "manual"])
        if random.random() > 0.6:
            self.alerts = [
                random.choice(["Перегрев", "Низкая влажность", "Ошибка насоса"])
            ]
        else:
            self.alerts = []

    def connect(self) -> Dict[str, Union[str, int]]:
        self.emulate()
        print(
            f"[LOG] Подключение к контроллеру выполнено, режим: {self.mode},
            количество тревог: {len(self.alerts)}"
        )
        return {
            "mode": self.mode,
            "alerts_count": len(self.alerts),
        }

class Database:
    def __init__(self, connection_string: str, storage_path: str):
        self.connection_string = connection_string
        self.storage_path = storage_path

```

```

def save_reading(self, sensor_id: int, value: float, timestamp: datetime):
    print(
        f"[LOG] БД: сохранено показание датчика {sensor_id} = {value} в
{timestamp}"
    )

def save_event(self, device_id: int, action: str, timestamp: datetime):
    print(
        f"[LOG] БД: сохранено событие устройства {device_id}: {action} в
{timestamp}"
    )

def get_history(
    self, start: datetime, end: datetime, device_id: Optional[int] = None
):
    print(
        f"[LOG] БД: запрос истории с {start} по {end} для устройства {device_id
if device_id else 'всех'}"
    )

    return []

def emulate(self) -> Dict[str, Union[str, int]]:
    return {
        "connection": "ok",
        "storage_path": self.storage_path,
        "records_today": random.randint(30, 300),
    }

def connect(self) -> Dict[str, Union[str, int]]:
    data = self.emulate()
    print(
        f"[LOG] Подключение к БД выполнено, записей за сегодня:
{data['records_today']}, путь: {data['storage_path']}"
    )
    return data

```

Листинг 3 — dashboard.html

```

<!DOCTYPE html>
<html lang="ru">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Умная теплица</title>
    <link rel="stylesheet" href="{ url_for('static', filename='css/main.css') }" >
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    <script src="{ url_for('static', filename='js/index.js') }" defer></script>
</head>

<body>
    <main class="container">
        <section class="card main">
            <h2>Агроном (оператор)</h2>
            <div class="grid">
                <article class="card">
                    <h2>Датчик температуры</h2>
                    <input id="temperature_value" type="text" readonly>
                </article>
                <article class="card">
                    <h2>Датчик влажности воздуха</h2>

```

```

        <input id="humidity_value" type="text" readonly>
    </article>
    <article class="card">
        <h2>Датчик влажности почвы</h2>
        <input id="soil_value" type="text" readonly>
    </article>
    <article class="card">
        <h2>Вентилятор</h2>
        <input id="fan_power" type="text" readonly>
    </article>
    <article class="card">
        <h2>Насос</h2>
        <input id="pump_power" type="text" readonly>
    </article>
    <article class="card">
        <h2>Контроллер</h2>
        <input id="controller_mode" type="text" readonly>
        <input id="controller_alerts" type="text" readonly>
    </article>
    <article class="card">
        <h2>База данных</h2>
        <input id="db_connection" type="text" readonly>
        <input id="db_records" type="text" readonly>
    </article>
</div>
</section>
</main>
</body>

</html>

```

Листинг 4 — main.css

```

:root {
    --font-family-base: Arial, sans-serif;

    --color-bg: #f4f8f5;
    --color-text: #182D4A;
    --color-muted: #4f6155;
    --color-surface: #ffffff;
    --color-surface-alt: #e3ecf7;
    --color-border: #b2c6e2;
    --color-accent: #182d4a;

    --radius-sm: 8px;
    --radius-md: 24px;

    --shadow-card: 0 2px 6px rgba(31, 43, 34, 0.08);
}

body {
    margin: 0;
    font-family: var(--font-family-base);
    background: var(--color-bg);
    color: var(--color-text);
}

.container {
    max-width: 600px;
    margin-inline: auto;
    padding: 24px;
}

```

```

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
  gap: 16px;
  margin-top: 18px;
}

.card {
  background: var(--color-surface-alt);
  border: 1px dashed var(--color-border);
  border-radius: var(--radius-sm);
  padding: 18px;
  box-shadow: var(--shadow-card);

  &>h2 {
    font-size: 15px;
    margin-top: 0;
    border-bottom: 1px solid var(--color-border);
    padding-bottom: 7px;
  }

  &>input {
    width: 100%;
    padding: 10px;
    border-radius: var(--radius-sm);
    border: 1px solid var(--color-border);
    font-size: 14px;
    background: var(--color-surface);
    box-sizing: border-box;

    &+& {
      margin-top: 8px;
    }
  }

  &.main {
    background: var(--color-surface);
    border: none;
    border-radius: var(--radius-md);

    &>h2 {
      font-size: 18px;
      border-radius: var(--radius-md) var(--radius-md) 0 0;
      background-color: var(--color-accent);
      padding: 18px;
      margin: -18px -18px 0;
      margin-bottom: 18px;
      color: var(--color-surface);
    }
  }
}

```

Листинг 5 — index.js

```

function getTemperatureData() {
  $.ajax({
    type: "GET",
    url: "/connect/temperature",
    dataType: "json",
    contentType: "application/json",
    data: {},

```



```

        success: function (response) {
            document.getElementById("temperature_value").value = response.value + " "
+ response.unit;
        }
    });
}

function getHumidityData() {
    $.ajax({
        type: "GET",
        url: "/connect/humidity",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            document.getElementById("humidity_value").value = response.value + " " +
response.unit;
        }
    });
}

function getSoilData() {
    $.ajax({
        type: "GET",
        url: "/connect/soil",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            document.getElementById("soil_value").value = response.value + " " +
response.unit;
        }
    });
}

function getFanData() {
    $.ajax({
        type: "GET",
        url: "/connect/fan",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            document.getElementById("fan_power").value = response.power + " Вт;
статус: " + response.status;
        }
    });
}

function getPumpData() {
    $.ajax({
        type: "GET",
        url: "/connect/pump",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            document.getElementById("pump_power").value = response.power + " Вт;
статус: " + response.status;
        }
    });
}

```

```

function getControllerData() {
    $.ajax({
        type: "GET",
        url: "/connect/controller",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            document.getElementById("controller_mode").value = "Режим: " +
response.mode;
            document.getElementById("controller_alerts").value = "Тревоги: " +
response.alerts_count;
        }
    });
}

function getDatabaseData() {
    $.ajax({
        type: "GET",
        url: "/connect/database",
        dataType: "json",
        contentType: "application/json",
        data: {},
        success: function (response) {
            document.getElementById("db_connection").value = "Соединение: " +
response.connection;
            document.getElementById("db_records").value = "Записей за сегодня: " +
response.records_today;
        }
    });
}

function updateDashboard() {
    getTemperatureData();
    getHumidityData();
    getSoilData();
    getFanData();
    getPumpData();
    getControllerData();
    getDatabaseData();
}

setInterval(updateDashboard, 1000);
updateDashboard();

```